

ROS2 Installation Guide

This is an overall guide for install ROS2 Humble on a computer, for editing and testing packages for your ROS bot. I have tested these on Windows and Ubuntu 24.04. Please share feedback on this document, so it can continue to be improved.

Option 1: Docker

Docker is my recommended option. Docker containers are lightweight, isolated development environments that contain all the software you need in a tidy package. They can only connect to the rest of your files if you give them permission. In my description below, I have you hook up a local directory

- *For Windows machines:* Install the Windows Subsystem for Linux (wsl). I recommend going to the Microsoft Store and installing Ubuntu.
- If using a Mac, no need for WSL. MacOS is a Unix system, so just about all of the functionality you would need from WSL is there behind the scenes using [Terminal](#). Can just start by installing Docker Desktop, as below.
- Download and install Docker Desktop: [Docker Desktop Install Page](#)
- In the Docker Desktop settings, you may need to select the option in [Resources/WSL Integration](#) called [Enable integration with my default WSL distro](#)
- I have included my [Dockerfile](#), which can be used to build a ROS workspace and configure it for yourself.
- You will then need to open Docker Desktop and wsl, and within wsl, navigate to the folder that has my Dockerfile. I recommend setting up a symbolic link in wsl's home directory, which is represented by the symbol ~. To navigate to a folder, use:

```
1 cd /path/to/folder/of/interest
```

- To create a symbolic link, use the following, I would put the symbolic folder in the home directory (~) at ~/robotics:

```
1 ln -s /full/path/to/folder/ /path/to/symbolic_folder/
```

- To build the Dockerfile, use the following command. It will grab the file named [Dockerfile](#) and create an image called `ros2-dev:latest`

```
1 docker build -t ros2-dev .
```

- You've now created an image! To run a container from this image, use:

```
1 docker run -it ros2-dev
```

- Sometimes, I want to run multiple terminals in the same container, then I use:

```
1 docker exec -it <container-name> /bin/bash
```

- The container name is randomly generated each time. If I'm only running one container, I use the `tab` key to autocomplete the container name. Otherwise, `docker ps` shows you the list of currently running containers with their names.
- To run docker interactively with GUI integration and a local folder with a path `/path/to/local_dir/` such as `/c/Users/username/Documents/docker_ws`, use:

```
1 docker run -it -v /tmp/.X11-unix:/tmp/.X11-unix \  
2 -v /path/to/local_dir:/mnt/docker_ws \  
3 -w /mnt/docker_ws \  
4 -e DISPLAY \  
5 -e WAYLAND_DISPLAY \  
6 -e XDG_RUNTIME_DIR \  
7 -e PULSE_SERVER \  
8 ros2-dev:latest
```

- The `-v` flag lets you connect a local directory to a directory within the Docker container, giving access to local files, etc. I use `/mnt/docker_ws` as my location with Docker, but call it what you want! The Docker container will lose its memory upon deletion, but if you connect it to a local directory, you will keep any files modified there.
- The `-w` flag starts the terminal in your `/mnt/docker_ws` workspace.
- The `-e` flags connect the container to the graphical interface of your computer. Haven't tested on a Mac yet. Reach out with feedback!
- I have included this command in the attached `startDocker.sh` script. You'll need to edit the `LOCAL_DIR` variable in this file to match your path on your computer.
- To use this shell script, navigate to the folder with it and type `./startDocker.sh` or `sh startDocker.sh`
- The long `docker run` command above needs some different flags in Linux, so I've also included `startDockerLinux.sh` for those devices.

Option 2: VirtualBox Install

Virtual machines are heavier than Docker, which is why Docker is my preferred tool here. However, they do give you full desktop functionality, which many may enjoy. Below are the instructions for using

a Virtual Machine with VirtualBox, but many virtual machine tools exist.

- Download Ubuntu 22.04 .iso file from <https://releases.ubuntu.com/jammy/>
- Download VirtualBox from <https://www.virtualbox.org/wiki/Downloads>. You will want the platform packages for a Windows (or MacOS) host, and the VirtualBox Extension Pack.
- Need to set up `sudo` rights for user. In the Terminal, run the following commands:

```
1 su -  
2 usermod -a -G sudo vboxuser
```

- Then, restart the virtualbox
- Need to configure copy-paste as well, go to `Machine/Settings/General/Features`, and check the `shared clipboard` option. This lets you copy and paste between Windows and your Virtual Machine.
- Useful to set up a shared folder that both Windows and the Virtual Machine can access.

Installing ROS2

- After installing the virtual machine, install ROS2 Humble, using the docs below. Just about anything you need can be installed via apt, using `sudo apt install ros-humble-something`
- Humble install docs: <https://docs.ros.org/en/humble/Installation/Ubuntu-Install-Debs.html>
- To start, you'll need to install both `ros-humble-desktop` and `ros-dev-tools`
- After install ROS, you'll need to `source` the ROS installation anytime you want to run `ros2` commands with:

```
1 source /opt/ros/humble/setup.bash
```

- It's a good idea to add this to the end of your `.bashrc` file, which is what runs anytime your BASH terminal opens. To do that, you can just run:

```
1 echo "source /opt/ros/humble/setup.bash" >> ~/.bashrc
```